



Crawlers

Recursive, Jev-powered repository investigation. Small functions follow context cues, judge each lead, and stop when evidence runs out.

DOCUMENT MAP

The specification moves from evidence to a gated experiment

Read pages 3-7 for the product case. Pages 8-31 define the architecture, contracts, experiment, verifier, and controls. Pages 32-36 define economics, evaluation, delivery, and product shape.

1. Executive summary	3	13. Jev judgment	20
2. Problem and latent demand	4	14. Calibration and failure behavior	22
3. Competitive position	6	15. Configuration	24
4. Product boundaries	7	16. MVP definition	26
5. System model	8	17. Verification step	27
6. Architecture	10	18. Developer experience	28
7. Public interface	12	19. Safety and privacy	30
8. Lead and evidence contracts	13	20. Cost and latency	32
9. Judgment and candidate contracts	14	21. Evaluation plan	33
10. Recursion engine	16	22. Delivery plan	34
11. Context expansion	18	23. Acceptance criteria	35
12. Context packer	19	24. Business and roadmap	36
		25. References	37

Reading rule: No output is called a bug until the separate verifier promotes it to a verified finding.

THE PRODUCT IN ONE PAGE

Crawlers is a precision experiment for team-scale verification

Crawlers is a local-first experiment that converts a starting signal into a bounded investigation graph for engineering teams. Mechanical expanders map blast radius across callers, configuration, tests, text references, and history. Jev judges which lead deserves another step.

Core promise: Give Crawlers a diff, failure, or suspicious location. Get back a short review queue split into verified findings and unverified leads, each with the chain that explains why it matters.

The unmet job

Teams can generate and change code faster than shared review capacity grows. The unmet need is fewer false positives and faster verification, not another stream of findings.

The mechanism

Small JSON-in/JSON-out functions perform one role each: seed, expand, judge, and report. A deterministic runner applies budgets, deduplication, ordering, policy, and resumability.

Jev's role

Jev is the experimental judgment kernel, not the author or actor. It scores branch value and recommends *expand*, *report*, *prune*, or *escalate*. Its probabilities are not assumed calibrated; deterministic code owns side effects.

First product

The blast-radius experiment starts from a pull request diff and follows affected symbols, call edges, configuration references, tests, and co-change history. It tests whether recursive routing improves precision without losing useful recall.

Recommended release shape

- **Standalone repository and CLI:** keep the product independent while preserving a future API surface.
- **Repo-local configuration:** commit `. crawlers / config . yaml`; keep credentials outside the repository.
- **Local execution first:** prove signal quality and safety before adding a hosted runner.
- **Review queue first:** no automatic issues, fixes, commits, or merges in the minimum viable product (MVP).

Success is trust, not scan volume

The MVP does not launch as a bug detector. It graduates from experiment only after held-out evaluation reaches at least 97% precision, beating the roughly 96% precision reported for leading AI reviewers [6], while preserving a declared recall floor and meeting cost and latency budgets. Until then, outputs are labeled *unverified leads*.

WHY NOW

Code production scaled; investigative attention did not

Evidence points to a team verification bottleneck: Sonar reports 42% of committed code is AI-generated or assisted and 96% of developers do not fully trust it; DORA links AI adoption with weaker delivery stability; a METR trial found experienced open-source developers 19% slower with early-2025 tools. [1-3] The product opportunity is fewer false positives, not more findings.

Primary users and jobs

USER	JOB TO BE DONE	CURRENT FAILURE
Review team	Understand a change's real blast radius	Attention is split across many PRs
Team using agents	Verify generated changes independently	Builder self-review repeats assumptions
Small team	Investigate failures without a dedicated security or reliability group	Manual search does not scale
Platform team	Turn incidents and telemetry into actionable repository evidence	Signals and code context stay separated

ALTERNATIVES AND BUYER

Existing tools leave room for an evidence-routed review queue

Static analyzers

Strong at known rules and deterministic checks.
Weak at contextual questions that cross symbols, configuration, history, and intent.

One-shot AI review

Useful for a first reading. It normally lacks an explicit investigation graph, recursive follow-up, explicit stop policy, and durable evidence chain.

Agentic coding loops

Good at producing a fix. Their builder context can bias verification, and broad autonomy can hide which evidence caused a conclusion.

Human investigation

Best available recursive reasoner. Also the most expensive and least scalable path for every uncertain lead.

Demand thesis

Crawlers targets teams with a shared review queue and measurable cost of false alarms. It sits after a signal exists but before a human commits to a full investigation, using cheap judgment on small branch decisions so human attention lands on evidence-rich paths.

Demand status: The verification gap is evidenced; willingness to pay for this exact shape is not. Validate with team design partners and measured reviewer acceptance. Solo developers may adopt a free CLI, but they are not the primary buyer.

A NARROW SURVIVING EDGE

The market already has deep code review; Crawlers must be different in form

Recursive or cross-file analysis is not novel by itself. Greptile markets autonomous repository-wide investigation and raised a \$25 million Series A. Semgrep Pro performs cross-file, interprocedural taint tracking. [4-5] The defensible experiment is narrower.

ALTERNATIVE	STRENGTH	CRAWLERS DISTINCTION
Greptile	Hosted autonomous review across a codebase graph	Repo-local, inspectable primitives; cost tied to judgments, not seats
Semgrep Pro	Deterministic cross-file security analysis	General defect hypotheses beyond authored static rules
CodeRabbit / Bugbot	Mature PR workflow and reported precision near 96%	Composable Unix interface, explicit graph, replayable typed decisions
Linters and tests	Fast deterministic truth for known properties	Use as evidence and verification, never replace them

Positioning

Nobody like this: a cheap, repo-local, Unix-style recursive investigation toolkit whose semantic branch decisions are typed Jev judgments and whose full graph can be inspected or replayed.

Precision bar

A third-party bug-recreation comparison reports 95.9% precision for a leading reviewer. [6] Crawlers should not claim a signal advantage until an independent held-out corpus shows at least 97% precision with a 95% confidence interval lower bound above 95%, plus a declared recall floor of 50%. If that bar is missed, the project remains an internal research tool.

Economic edge and risk

Jev unit cost is tiny, but low inference cost does not create pricing power. The category is moving toward bundled and free review. The initial business thesis is team-priced workflow value: reproducibility, local control, and evidence-chain review time saved. Price testing must happen after precision, not before it.

PRINCIPLES

The product earns trust by being small, bounded, and inspectable

1. One function, one job

Each primitive has one stable contract and composes through newline-delimited JSON (NDJSON). Any stage can be replaced without rewriting the pipeline.

2. Evidence before inference

A node cannot become a verified finding unless it cites repository evidence and passes a separate verifier. A model conclusion without a location or artifact is an unverified lead, never a reportable bug.

3. Recursion has a budget

Depth, nodes, elapsed time, bytes, and judgment calls are hard limits. The runner stops deterministically even if the model keeps asking to expand.

4. Policy is outside the model

Jev emits typed judgments and probabilities. Code applies thresholds, budgets, safety rules, and side effects. A prompt never grants authority.

5. Uncertainty is useful output

A low or unstable model probability does not become false certainty. The system can prune, request a stronger cue, or route a compact investigation packet to a reviewer.

6. Local first, hosted later

Repository extraction and graph building run locally in the MVP. A hosted option must accept minimized context, not require source upload by default.

7. Precision before breadth

A crawler that returns three verified findings is better than one that returns thirty speculative warnings. New expanders stay opt-in until evaluated.

8. Resumable by construction

Every crawl persists its manifest, queue, node states, judgments, and budget ledger so a stopped run can continue without repeating work.

In scope for v0.1

- + Local CLI and TypeScript SDK
- + Git repository ingestion
- + Diff-seeded blast-radius crawler
- + Four composable primitives
- + Jev question sets and policy routing
- + JSON, Markdown, and terminal reports
- + Offline replay of recorded judgments

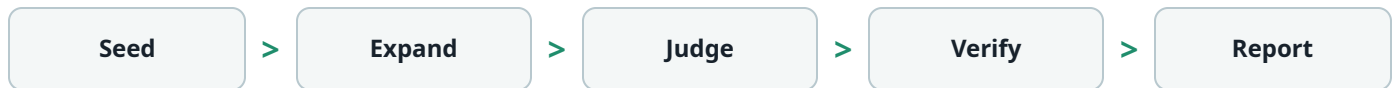
Explicitly out of scope

- ✗ Autonomous code modification
- ✗ Automatic issue or pull-request creation
- ✗ General vulnerability scanning claims
- ✗ Binary analysis or dynamic fuzzing
- ✗ Cross-tenant hosted source retention
- ✗ Replacing tests, linters, or human review
- ✗ Learning from private code across customers

THE INVESTIGATION GRAPH

A crawl is a graph, not a conversation

Every input, expansion, judgment, and output is represented as a durable typed record. The graph is the product's source of truth; Jev calls are replaceable decisions attached to its nodes.



Core entities

ENTITY	PURPOSE	IDENTITY
CrawlRun	One bounded investigation, configuration snapshot, and budget ledger	Generated run ID
Lead	A location or artifact that may warrant investigation	Canonical repository key
Edge	Why one lead produced another	Parent, child, cue type
Evidence	Exact material used to judge a lead	Content hash plus locator
Judgment	Typed Jev answers, probabilities, routing result, and model metadata	Lead plus question-set version
Candidate / VerifiedFinding	A report-routed hypothesis / a verifier-promoted result	Fingerprint of claim and evidence path

LIFECYCLE AND CONTROL

Immutable states and policy boundaries make the graph replayable



Terminal states are immutable. A resumed run processes only queued or retryable nodes. A new question-set version creates a new judgment record rather than mutating the old one.

Control plane and data plane

Deterministic control plane

Queue ordering, deduplication, budget checks, path normalization, redaction, caching, policy evaluation, persistence, and sink permissions.

Jev judgment plane

Bug likelihood, relevance, next cue, severity, and model probability. It receives minimized evidence and returns typed answers only.

Invariant: The same stored graph and judgment records must replay to the same routing outcome without another model call.

COMPONENTS AND TRUST BOUNDARIES

A stateful driver is the architecture; the four commands are its interfaces

Repository adapter

Reads Git, files, symbols, tests, and history. Enforces ignore rules and creates stable locators.

Seed registry

Turns a diff, file, error, or external reference into normalized initial leads.

Expander registry

Follows one cue type at a time: call graph, imports, config, tests, history, or data boundary.

Jev adapter

Builds minimized state, calls a versioned question set, validates output, and records usage.

Policy engine

Evaluates ordered predicates over typed answers. On error or contradiction, routes to review.

Graph store

Persists runs, nodes, edges, evidence hashes, judgments, candidates, verified findings, and checkpoints.

Stateful driver

Owns the frontier, visited set, checkpoints, retries, and run lifecycle. It invokes the four command interfaces rather than hiding state inside shell pipes.

Reporter

Groups duplicate review records and renders terminal, JSON, Markdown, and later review-queue output.

Audit logger

Records decisions without secrets or unnecessary source. Supports replay and evaluation.

DEPLOYMENT AND PACKAGE SHAPE

The same contracts must work locally, in CI, and later in hosted workers

MODE	REPOSITORY ACCESS	JEV ACCESS	TARGET RELEASE
Local CLI	Direct filesystem	Outbound API call	v0.1 default
CI runner	Checked-out workspace	Secret-managed API call	v0.2
Hosted worker	Scoped integration or uploaded bundle	Service-side	Post-validation

Repository layout

```
packages/  
  cli/      # crawlers and crawl-* commands  
  core/     # graph, scheduler, budgets, policy  
  adapters-git/ # repository and history extraction  
  adapter-jev/ # typed evaluation client  
  crawlers/  # blast-radius definition and future recipes  
  schemas/   # versioned JSON Schemas  
  question-sets/ # reviewable Jev questions and routes  
  examples/  # small, reproducible fixtures
```

The layout is normative at the package boundary, not at the folder-name level. Existing repository conventions win if the implementation home already defines them.

UNIX PRIMITIVES

Four commands form the stable product surface

COMMAND	CONSUMES	EMITS	RESPONSIBILITY
<code>crawl-seed</code>	Source descriptor	Lead	Create initial leads
<code>crawl-expand</code>	Lead	Lead + Edge	Follow context cues
<code>crawl-judge</code>	Lead + evidence	Judgment	Classify and score
<code>crawl-report</code>	Graph records	Review views	Deduplicate and render

Convenience orchestrator

`crawlers run` is the stateful driver. It owns the frontier, visited set, checkpoints, policy, and budgets while invoking the same four interfaces exposed as commands. Shell composition is an interoperability promise, not a claim that recursion is stateless.

```
# Full MVP run
crawlers run blast-radius \
  --base origin/main --head HEAD \
  --format terminal --budget-profile ci

# Compose one step at a time
crawl-seed diff --base origin/main --head HEAD \
  | crawl-expand --cue auto \
  | crawl-judge --set bug-node-v1 \
  | crawl-report --format ndjson

# Resume without repeating completed judgments
crawlers resume cr_01J... --format markdown
```

Stream contract

- **Standard input:** UTF-8 NDJSON, one versioned record per line.
- **Standard output:** data only. Diagnostics and progress go to standard error.
- **Ordering:** stable within a run. Every record carries `runId`, `sequence`, and `schemaVersion`.
- **Backpressure:** stages read incrementally and flush each complete record.
- **Exit status:** 0 success, 1 completed with review items, 2 policy stop, 3 infrastructure or schema error, 64 usage error.
- **Compatibility:** readers accept the current major schema and one previous major. Unknown fields are ignored; unknown record types fail closed.

Composition rule: Every primitive must remain useful when called alone. If a feature requires hidden orchestration state, it does not belong in a primitive.

LEAD AND EVIDENCE

Stable records make recursion explainable and replayable

Lead

```
{
  "type": "lead",
  "schemaVersion": "1.0",
  "runId": "cr_01J...",
  "leadId": "ld_01J...",
  "canonicalKey": "git:abc123:src/pay.ts:capturePayment:function",
  "kind": "symbol",
  "locator": {
    "repo": "local",
    "commit": "abc123",
    "path": "src/pay.ts",
    "symbol": "capturePayment",
    "lines": [41, 78]
  },
  "origin": { "kind": "diff", "parentLeadId": null },
  "depth": 0,
  "priority": 0.82,
  "cueHints": ["callers", "config", "tests"]
}
```

Required lead properties

- **Canonical identity:** The key is revision + normalized file + symbol + lexical scope. Line numbers are locators, not identity. A run-level visited set rejects the same key before expansion or judgment.
- **Typed kind:** file, symbol, config, test, error, dependency, or boundary.
- **Provenance:** Every non-root lead names the parent and cue that produced it.
- **No raw secrets:** Locators may name files; evidence extraction applies redaction and deny rules before any remote call.

Evidence

```
{
  "type": "evidence",
  "evidenceId": "ev_01J...",
  "leadId": "ld_01J...",
  "kind": "source_excerpt",
  "locator": { "path": "src/pay.ts", "lines": [52, 67] },
  "contentHash": "sha256:...",
  "text": "...minimized excerpt...",
  "redactions": [{ "class": "credential", "count": 1 }]
}
```

Retention default: Store evidence metadata and hashes. Raw excerpts are run-local and expire with the run unless the user explicitly enables durable storage.

JUDGMENT AND CANDIDATE

A report route creates a candidate, not a verified finding

Judgment

```
{
  "type": "judgment",
  "schemaVersion": "1.0",
  "leadId": "ld_01J...",
  "questionSet": "bug-node-v1",
  "questionSetDigest": "sha256:...",
  "answers": {
    "action": { "choice": "report", "probability": 0.91 },
    "bugLikely": { "value": true, "probability": 0.87 },
    "severity": { "score": 2.2 },
    "nextCue": { "choice": "caller", "probability": 0.74 },
    "expandValue": { "score": 2.4 }
  },
  "route": "report",
  "latencyMs": 984,
  "usage": { "inputTokens": 742 }
}
```

CANDIDATE AND PROMOTION

A candidate stays unverified until a separate check succeeds

```
{
  "type": "candidate",
  "candidateId": "cd_01J...",
  "fingerprint": "sha256:...",
  "title": "Caller can submit an unlocked quote",
  "classification": "correctness",
  "severity": "high",
  "bugProbability": 0.87,
  "primary": { "path": "src/pay.ts", "lines": [52, 67] },
  "claim": "capturePayment accepts a quote without rechecking its lock.",
  "evidenceChain": ["ld_root", "ld_caller", "ld_target"],
  "verificationArtifact": {
    "kind": "failing_test_proposal",
    "check": "Call capturePayment with an expired lock.",
    "verified": false
  },
  "status": "unverified_lead"
}
```

Candidate invariants

- **Specific:** Names one defect claim, one primary location, and one observable consequence.
- **Traceable:** Every claim points to stored evidence IDs and a parent chain back to a root seed.
- **Falsifiable:** Includes a reproducer sketch, failing-test proposal, or input trace. A verifier step checks the artifact before the report can use the label *verified finding*.
- **Non-duplicative:** Fingerprint combines defect class, primary location, and normalized evidence path.
- **Honest status:** An unchecked artifact is an `unverified_lead`, never a bug. MVP output mutates no repository or remote system.

Run manifest

The manifest snapshots crawler version, repository commit, configuration digest, question-set digests, ignore rules, budget limits, environment metadata, and final counters. It is sufficient to explain what ran without retaining source content.

BOUNDED SEARCH

The runner follows the strongest evidence path, not every edge

Scheduling algorithm

1. Normalize and deduplicate all seeds.
2. Score queued leads using the parent's Jev `expandValue`, plus deterministic cue reliability, diff proximity, depth penalty, novelty, and estimated cost.
3. Pop the highest-value lead. Stop first if any hard budget is exhausted.
4. Collect the smallest evidence bundle required by its kind and allowed cues.
5. Run deterministic checks. If a check proves the node safe or invalid, prune without a Jev call.
6. Call Jev, validate the typed response, and apply ordered policy routes.
7. On `expand`, run only the selected cue or top bounded cue set, then enqueue unseen children.
8. On `report`, create a candidate and run the verifier. Only a successful falsifiable artifact becomes a verified finding; otherwise retain an unverified lead.
9. Checkpoint after every terminal node and continue until the queue is empty or a stop condition fires.

PRIORITY AND TERMINATION

Graded value orders the frontier; deterministic rules stop the run

```
priority =  
  0.35 * judge_expand_value  
+ 0.20 * cue_reliability  
+ 0.18 * diff_proximity  
+ 0.12 * severity_hint  
+ 0.15 * novelty  
- 0.12 * depth  
- 0.08 * normalized_cost
```

Weights are initial defaults, not learned truth. They are configuration and must be tuned against recorded runs. Deterministic tie-break: canonical key, ascending.

Hard stop conditions

Budget exhausted

Any limit on nodes, Jev calls, input tokens, elapsed time, or estimated cost is reached.

Depth reached

A child would exceed `maxDepth`. It is recorded as unexpanded, not silently dropped.

Queue drained

No unseen canonical keys remain in the frontier after visited-set filtering. The engine stops without a model call.

Diminishing returns

A loop-continue style gate scores recent yield. Stop when three consecutive batches produce no verified finding and declining mean expand value. Contradiction defaults to stop.

Safety stop

Redaction failure, repository drift, schema mismatch, or contradictory policy output ends the run in review.

Default traversal: Best-first search with a depth penalty and explicit visited set. The judge supplies a graded expansion value, not a binary expand/prune switch; policy still decides whether a child may enter the frontier.

CUE REGISTRY

Each cue is a deterministic relationship with a known cost

CUE	PRODUCES	DEFAULT CAP	RELIABILITY
<code>callers</code>	Symbols that invoke a changed symbol	12	High
<code>callees</code>	Symbols directly invoked by the lead	8	Medium
<code>imports</code>	Direct importers and imported modules	10	Medium
<code>config</code>	Configuration that references the symbol, route, flag, or key	8	High
<code>tests</code>	Tests naming or covering the lead	10	High
<code>cochange</code>	Files historically changed with the lead	6	Low until evaluated
<code>dataflow</code>	Input, validation, trust, persistence, and output boundaries	10	Medium
<code>runtime</code>	Matching stack frames, logs, or trace locations	8	High

Expander interface

```
interface Expander {
  name: string;
  version: string;
  supports(lead: Lead, repo: RepoCapabilities): boolean;
  estimate(lead: Lead): { children: number; bytes: number };
  expand(input: ExpandInput): AsyncIterable<LeadEdgePair>;
}
```

Context selection rules

- **Prefer structure:** Use parser or language-server relationships before text search. Record the fallback when structure is unavailable.
- **Preserve direction:** A caller edge is not interchangeable with a callee edge. Direction is part of the evidence path.
- **Bound fan-out:** Rank candidates deterministically, emit only the cap, and record how many were omitted.
- **Carry cues, not conclusions:** Expanders state the relationship. They do not declare a bug.
- **Language adapters:** v0.1 supports TypeScript and JavaScript first. Generic text and Git-history expanders work for any repository.

False-confidence risk: An incomplete call graph can look complete. Every structural edge carries its extraction method and a completeness grade: `complete`, `partial`, or `unknown`.

BOUNDED MODEL STATE

Truncation is explicit, ordered, and visible in every judgment

The context packer turns deterministic repository evidence into the bounded state sent to Jev. It targets 1,800 to 2,300 input tokens per node and must never rely on silent string truncation.

Packing order

1. Question-set instructions and typed criteria.
2. Lead identity, root intent, cue type, depth, and remaining budget.
3. Primary changed hunk or runtime frame.
4. Direct parent evidence that caused this expansion.
5. Selected caller, callee, test, configuration, or text-reference excerpts.
6. Path summaries for earlier ancestors.
7. Optional history metadata and low-reliability context.

Truncation order

When the state would exceed the configured 6,000-character cap, drop in reverse priority: history prose, ancestor summaries beyond the parent, low-reliability neighbors, unchanged surrounding lines, then secondary excerpts. Never truncate question criteria, lead identity, the primary hunk, redaction markers, or the fact that material was omitted.

```
"packing": {
  "capChars": 6000,
  "inputChars": 8421,
  "packedChars": 5934,
  "omitted": [
    { "class": "history", "count": 3 },
    { "class": "secondary_excerpt", "count": 2 }
  ],
  "completeness": "partial"
}
```

Untrusted spans

Comments, string literals, log messages, and documentation are either excluded or wrapped as clearly delimited untrusted evidence. Instruction-like text inside them cannot modify questions, policy, tools, budgets, or sinks. The Jev request is typed and tool-free.

Routing consequence: A report recommendation with `completeness=partial` cannot become a verified finding until the verifier obtains the omitted artifact or reproduces the defect independently.

TYPED DECISIONS

Jev supplies unproven branch scores; deterministic policy limits their effect

Question set: `bug-node-v1`

QUESTION	TYPE	MEANING
<code>action</code>	Choice	expand, report, prune, escalate
<code>bugLikely</code>	Boolean	Evidence supports a real behavioral defect
<code>severity</code>	Score 0-3	Impact if the hypothesis is correct
<code>evidenceSufficient</code>	Boolean	Current evidence can support the selected action
<code>nextCue</code>	Choice	Best available mechanical relationship
<code>expandValue</code>	Score 0-3	Expected value of spending one more node on this branch

Choice criteria

- **Expand:** A concrete uncertainty remains and one named cue is likely to resolve it.
- **Report:** The evidence supports a specific defect, consequence, and location.
- **Prune:** The lead is explained, irrelevant, duplicate, or lacks a credible defect path.
- **Escalate:** The lead may matter, but required context is unavailable, restricted, or ambiguous enough that automated routing is unsafe.

POLICY ROUTING

Ordered predicates convert typed scores into bounded actions

```
candidate if action=report
           and P(bugLikely) >= 0.85
           and P(evidenceSufficient) >= 0.80
           then send to verifier

expand    if action=expand
           and P(action) >= 0.70
           and expandValue >= 1.5
           and depth < maxDepth

prune     if action=prune
           and P(action) >= 0.75

escalate  if severity >= 2.5
           and P(bugLikely) >= 0.60

review    otherwise, including scorer error
```

Policy order matters: Escalation is evaluated before pruning, so a potentially severe but uncertain lead cannot disappear because another answer weakly favored prune.

State passed to Jev

Only normalized intent, lead metadata, a packed evidence excerpt, parent path summaries, extraction quality, and remaining budget are included. Secrets, unrelated files, raw environment values, comments or strings that cannot be safely delimited, and prior natural-language chain-of-thought are excluded. Jev has no tools.

CALIBRATION AND FAILURE BEHAVIOR

Jev has no code-bug validation yet, so v0.1 is an experiment

An internal decision-workflow evaluation recorded 17 September 2026 does not justify calling its probabilities calibrated. One boolean was systematically underconfident, with a 0.12 mean prediction against a 0.69 observed safe rate; `safe_to_automerge` had zero recall at a 0.5 threshold; `needs_human` was broadly conservative. No code-bug evaluations exist.

Calibration corpus

Mine revert commits and later bug-fix commits from Git history, reconstruct the pre-fix change, and label the broken and corrected variants. Add safe controls, misleading cues, missing context, duplicate paths, and severe ambiguous cases. The two anecdotal project scars may inform cue design, but they do not validate a code crawler.

Route accuracy

Confusion matrix across report, expand, prune, and escalate. A safe change routed to report is a precision defect; a known bug routed to prune is a recall defect.

Probability reliability

Bucket predicted probabilities against observed labels. Until reliability is demonstrated, probabilities are ranking signals only, not calibrated confidence.

Wording sensitivity

Change one instruction at a time, rerun the fixed corpus, and keep the previous set available for comparison and replay.

Repository holdout

Do not tune and report on the same repository. At least one held-out codebase is required before a release claim.

FAILURE BEHAVIOR AND VERSIONING

Errors remain visible and route toward review

FAILURE	RUNNER BEHAVIOR	USER-VISIBLE RESULT
Jev timeout	One bounded retry, then review	Unjudged node retained
Invalid schema	No retry on same response	Infrastructure error
Contradictory answers	Conservative ordered route	Review packet
Budget exhausted	Stop new work, flush review records	Partial result with coverage note
Repository changed	Freeze run and invalidate locators	Resume requires new run
Redaction failure	Block remote call	Safety stop

Versioning

Question sets are immutable once released. Their digest is stored in every judgment. A revised question set receives a new semantic version and runs through the same development, validation, and held-out sets. Policies are versioned separately because the same answers may be routed differently.

Never: Treat Jev output as permission to alter code, create a remote issue, disclose repository content, or exceed a configured budget.

REPO-LOCAL RECIPE

Configuration is committed; credentials and policy overrides are not

```
# .crawlers/config.yaml
version: 1
crawler: blast-radius
languages: [typescript, javascript]

seed:
  type: diff
  base: origin/main

expanders:
  callers: { enabled: true, maxChildren: 12 }
  callees: { enabled: true, maxChildren: 8 }
  config: { enabled: true, maxChildren: 8 }
  tests: { enabled: true, maxChildren: 10 }
  cochange: { enabled: false, maxChildren: 6 }

budget:
  maxDepth: 4
  maxNodes: 250
  maxJevCalls: 180
  maxElapsedSeconds: 300
  maxEstimatedCostUsd: 0.02

report:
  formats: [terminal, json]
  minimumSeverity: medium
  sink: review

privacy:
  deny: ["**/.env*", "**/*secret*", "**/*.pem"]
  retainEvidence: false
```

PRECEDENCE AND SECRETS

Repository settings can narrow behavior but cannot bypass policy

1. Built-in safe defaults.
2. Repository configuration.
3. Named budget profile such as `local` or `ci`.
4. Command-line reductions only. The CLI may tighten budgets but cannot exceed repository maximums without an explicit unsafe flag.
5. Organization policy, when present, can only restrict behavior.

Secrets

The Jev credential comes from the approved runtime credential mechanism or process environment. Configuration stores only the provider and model identifier. The CLI must reject secret-looking values in committed configuration.

Ignore semantics

Crawlers honors `.gitignore` first, then applies `.crawlersignore` and privacy denies. Explicit seed paths cannot bypass privacy denies. Generated, vendored, minified, binary, and lock files are excluded by default unless a crawler specifically owns them.

Design decision: Repo-local configuration plus a local runner is the v0.1 default. A hosted engine remains compatible because the same manifest and schemas cross the boundary.

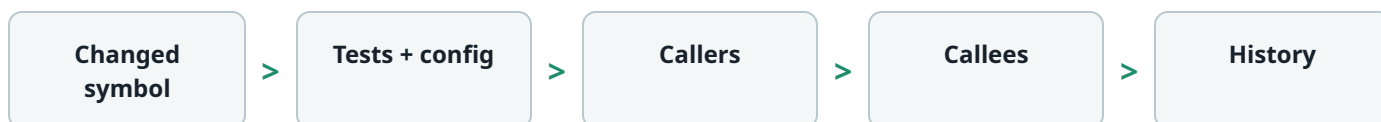
BLAST-RADIUS CRAWLER

The first crawler answers one question: what did this change put at risk?

Input

A Git diff between two immutable revisions. The seed stage parses touched files and hunks, maps changed lines to symbols where possible, and emits one root lead per changed symbol or top-level region.

Expansion order



- **Tests and configuration first:** They often reveal the intended contract and hidden behavior switches with low fan-out.
- **Callers next:** They show which assumptions the changed interface may violate.
- **Callees after callers:** They matter when the change alters input shape, ordering, error handling, or side effects.
- **Co-change history last:** It is useful but noisy, so it remains off by default until evaluated.

Defect classes for v0.1

Contract mismatch

A caller, callee, or test assumes behavior the diff changed without coordinated updates.

Missing guard

A changed path bypasses validation, authorization, state, or idempotency checks.

Configuration drift

Runtime configuration, feature flags, generated files, or deployment metadata no longer match code.

Incomplete migration

One representation, route, schema, vocabulary, or compatibility path was updated while another was not.

Error-path regression

The change handles success but drops cleanup, retries, status propagation, or recovery behavior.

Test blind spot

An affected behavior has no meaningful assertion or the existing test proves the wrong condition.

Output

A ranked review queue of verified findings and unverified leads, pruned-node statistics, unexpanded-lead counts, and one coverage note that states which adapters or budgets limited the run.

FALSIFIABLE ARTIFACTS

The judge can propose a defect; only a verifier can promote it

Every report route must include one falsifiable artifact. Verification is a separate, deterministic or sandboxed step with its own budget. It tests the claim rather than asking the same model to agree with itself.

ARTIFACT	MINIMUM CONTENT	VERIFIER
Reproducer sketch	Inputs, preconditions, action, expected and observed result	Existing test harness or isolated command
Failing-test proposal	Target test file, assertion, fixture inputs	Temporary generated test in sandbox; never committed
Input trace	Source, transformations, missing guard, sink	Static path checker or reviewer confirmation

Promotion rules

- **Verified finding:** The artifact fails on the suspect revision and passes on the corrected or baseline revision, or a deterministic checker confirms the input trace.
- **Unverified lead:** The artifact is plausible but cannot run, lacks a required fixture, reaches a partial graph, or produces an ambiguous result.
- **Rejected lead:** The artifact does not reproduce the claim or contradicts the cited evidence.

Sandbox rules

The verifier runs read-only by default. A temporary test may write only inside an isolated worktree and is removed after the run. No network, secrets, production services, database writes, package installation, or repository commit is allowed unless a later explicitly permissioned verifier owns that capability.

Report language

Only verified findings may use the label *bug*. All other surfaced items use *unverified lead* and show exactly what prevented verification. The terminal and JSON schemas preserve this distinction.

Precision accounting: Product precision is calculated only from surfaced verified findings. Unverified leads are tracked separately as reviewer load and cannot be hidden from the acceptance report.

THE REVIEW QUEUE

Every surfaced item should be reviewable in under five minutes

Terminal view

```
VERIFIED HIGH Caller can submit an unlocked quote
src/pay.ts:52-67

Claim
  capturePayment accepts a quote after its lock expires.

Why this surfaced
  diff: quote lock check moved
  -> caller: submitOrder still calls capturePayment directly
  -> config: lock TTL remains enabled in production

Verify
  Call submitOrder with a quote whose lockedUntil is in the past.

Verification
  failing_test | failed on suspect | passed on baseline

Route
  review | verified finding fd_01J... | run cr_01J...
```

REPORT CONTRACT

The review record exposes evidence, limits, and verification state

- **Claim:** One sentence describing the suspected defect and consequence.
- **Primary location:** Repository-relative file and immutable line range.
- **Evidence chain:** Root signal, each cue followed, and the evidence that changed the route.
- **Model probability and severity:** Separate values. The probability is a ranking signal until code-bug reliability is demonstrated.
- **Verification artifact:** Reproducer sketch, failing-test proposal, or input trace, plus its observed result.
- **Coverage note:** Missing adapters, omitted fan-out, budget stops, and partial extraction quality.

Deduplication and grouping

Findings with the same primary location and normalized claim are grouped. The strongest evidence path becomes primary; alternate paths remain visible. Repeated findings across runs keep a stable fingerprint so CI can mark them unchanged, resolved, or regressed.

Suppression

Suppressions are explicit records with fingerprint, reason, author, creation time, and optional expiry. Path-wide or rule-wide suppressions require a reason. Jev probability alone never suppresses a deterministic scanner finding.

Sink policy

v0.1

Terminal, JSON, Markdown file, and process exit status. Verified findings and unverified leads are separate queues; both require human review.

Later

Pull-request annotations, issue trackers, and dashboards. Each is opt-in and separated from discovery by a permissioned sink.

TRUST REQUIREMENTS

Source minimization and side-effect isolation are release gates

Threat model

RISK	REQUIRED CONTROL
Secret leakage in evidence	Deny rules, entropy and key-name detection, redaction before network access
Prompt injection in source	Treat repository text as evidence, never instruction; fixed question sets only
Runaway recursion	Hard budgets enforced before enqueue and before model call
Model-induced side effect	Judgment adapter has no write capability; sinks are separate and permissioned
Stale or shifted code	Immutable revision locators; stop when worktree identity changes
Cross-run data exposure	Per-run storage namespace; no shared private-code learning
Misleading certainty	Expose probabilities, extraction quality, omitted context, and review route

DATA HANDLING AND PERMISSIONS

Retention, credentials, CI, and sinks fail closed

- Read-only repository access.
- Request zero data retention only after the active plan and route are verified; never infer enforcement from a request flag.
- Minimized source excerpts, not whole files.
- A pre-judge redaction pass strips secrets and tokens. Secret-bearing configuration is excluded from both evidence and expansion scope.
- Run-local evidence deleted at configured expiry; manifests and hashes may remain.
- Telemetry off by default for local runs. Opt-in telemetry contains counts and timings, not source or paths.
- Each gateway key has a hard budget and spend alerts. Shared unrestricted keys are not accepted.

Permissions

The core runner can read the working tree and write only inside its run directory. Network access is limited to the configured judgment provider. GitHub Actions are pinned to full commit SHAs. Jobs triggered by fork pull requests never receive secrets and therefore skip remote judgment. Report sinks are separate and disabled unless explicitly selected.

Prompt injection rule: Comments, documentation, filenames, test fixtures, and fetched error text cannot change the crawler's task, question set, budget, ignore rules, or destination. They are untrusted input.

Auditability

Before first use and on provider changes, a smoke test requests the privacy policy and asserts on returned `planningReasoning` metadata rather than assuming the flag worked. Vercel documents team-wide zero data retention as available only on Pro and Enterprise plans, so an unverified plan is treated as ordinary retention. [7] Every judgment records state and question-set digests, response, route, latency, and usage.

HONEST ECONOMICS

Inference is cheap; context size and reviewer trust dominate the economics

Jev cost model

Published Jev input pricing is \$0.042 per million tokens, with output currently unmetered. [8] A realistic node carries the question set plus 1,800 to 2,300 input tokens of context, or about \$0.00008 to \$0.00010 per judgment. Measured calls in the current evaluation were \$0.000055 and \$0.000092.

SCENARIO	ASSUMPTION	ESTIMATED JEV COST
Compact 500-node crawl	State stays near 6,000-character cap	\$0.04-\$0.05
Context-heavy crawl	2,000-4,000 judged nodes near 2,000 input tokens each	\$0.17-\$0.34
Deterministic-only branch	No semantic judgment needed	\$0

Economic constraints

- **Expansion stays mechanical:** Abstract syntax trees, language services, grep, and Git history perform expansion. A model-powered expander would multiply cost and latency at the highest-fan-out stage.
- **Cache by evidence digest:** Identical lead, evidence, question set, and policy version reuse the recorded judgment.
- **Two-tier future:** If semantic expansion ever requires a larger model, expose it as an opt-in premium tier with a separate budget and price. Do not hide it inside base crawl cost.
- **Pricing risk:** Competitors can bundle review into existing seats. The economic moat must be review time saved, portability, and control, not token arbitrage.

PR latency budget

The default CI profile targets a five-minute wall-clock limit. Deterministic extraction runs concurrently by cue. Jev judgments run in a bounded worker pool of 16, subject to provider limits and the spend ledger. Verification jobs run in a pool of 4 because tests are heavier.

30 s

seed, parse, and index target

180 s

expand and judge target

90 s

verify and report target

FROM EXPERIMENT TO PRODUCT

Git history supplies the labeled bugs that anecdotes cannot

Corpus construction

- 1. Identify revert commits and commits whose messages or linked changes describe a bug fix.
- 2. Confirm each candidate by reading the diff and surrounding test or issue evidence. Exclude ambiguous refactors.
- 3. Reconstruct the parent revision as the suspect state and the fix as ground truth.
- 4. Create a matched safe control by applying the coordinated fix or selecting a nearby non-bug change.
- 5. Freeze repository revision, root diff, expected defect location, acceptable evidence paths, and severity label.
- 6. Split by repository into development, validation, and held-out test sets to prevent leakage.

Primary metrics

METRIC	DEFINITION	GRADUATION GATE
Precision	Accepted verified findings / all verified findings	$\geq 97\%$; lower 95% CI $> 95\%$
Recall	Known bugs surfaced / eligible known bugs	$\geq 50\%$ declared floor
Unverified load	Unverified leads per evaluated PR	≤ 0.25 median
Time to verdict	Wall time from seed to report	p95 ≤ 5 minutes
Reviewer effort	Median time to accept or reject	≤ 5 minutes per item
Cost	Provider spend per PR	p95 $\leq \$0.10$ compact profile

Ablations

Compare flat one-shot review, deterministic expansion plus one final judgment, and full per-node recursive judgment. Also remove each cue family in turn. The product thesis survives only if recursion improves precision or recall enough to justify added latency and complexity.

Rollout boundary

Dogfood first: Run only on repositories the team owns. Do not process anyone else's code until redaction, provider-retention verification, spend controls, prompt-injection handling, and the held-out evaluation gates are complete.

FOUR GATED PHASES

Build the measurement system before claiming a bug product

Phase 0: corpus and harness

- + Mine and label fix/revert cases plus safe controls
- + Define precision, recall, reviewer-load, latency, and cost reports
- + Freeze schema, replay format, and question-set versioning
- + Gate: corpus review and deterministic replay pass

Phase 1: deterministic graph

- + Diff seeder; TypeScript/JavaScript symbol adapter
- + Callers, callees, tests, configuration, text-search, and Git-history expanders
- + Canonical identity, visited set, best-first frontier, hard budgets, checkpoints
- + Context packer, truncation ledger, and pre-judge redaction
- + Gate: graph fixtures prove bounded, repeatable expansion with no provider call

Phase 2: Jev experiment

- + `bug-node-v1` questions, versioned policy, cache, and audit records
- + Probability reliability plots and route confusion matrix
- + Diminishing-return stop gate and infrastructure failure paths
- + Gate: validation-set thresholds chosen before held-out execution

Phase 3: verifier and local CLI

- + Reproducer, failing-test, and input-trace artifact contracts
- + Sandboxed verifier with no network or persistent writes
- + Terminal, JSON, and Markdown reports with honest status labels
- + Gate: held-out metrics meet graduation criteria

Phase 4: team pilot

- + SHA-pinned CI workflow; fork-safe secret behavior
- + Bounded parallel pools, PR latency budgets, gateway key quotas and alerts
- + Review-outcome capture from a small set of owned repositories
- + Gate: four weeks of stable precision and reviewer acceptance before external beta

Kill criteria: Stop or reposition if recursion does not beat the flat baseline, if verified precision stays below 97%, or if review time saved does not exceed the added operational complexity.

DEFINITION OF DONE FOR V0.1

The local experiment is complete only when behavior is measurable end to end

Functional

- + Seeds any valid Git revision pair.
- + Builds typed symbol and file leads.
- + Expands all enabled mechanical cues.
- + Deduplicates by revision, file, symbol, and scope.
- + Uses a graded judge score to rank the frontier.
- + Stops on every configured budget.
- + Stops on measured diminishing returns.
- + Resumes without repeated judgments.
- + Requires a falsifiable artifact on report routes.
- + Separates verified findings from unverified leads.

Safety and operations

- + Redaction runs before every remote call.
- + Secret-bearing config never enters expand scope.
- + Provider-retention smoke test records a pass or blocks remote use.
- + Fork PRs run without provider secrets.
- + GitHub Actions are pinned by full SHA.
- + Gateway key budget and spend alert are configured.
- + Untrusted spans cannot alter questions or policy.
- + Every judgment is replayable from stored metadata.
- + PR profile completes within the five-minute target or reports a budget stop.
- + No command can commit, push, file an issue, or merge.

Quality

- + Development, validation, and held-out repositories are disjoint.
- + Evaluation reports confidence intervals and the count behind every percentage.
- + Precision, recall, unverified load, reviewer effort, latency, and cost are all reported together.
- + No public bug-detection claim ships before the held-out graduation gate passes.
- + Every code path that calls Jev has timeout, retry, schema-error, and spend-stop tests.

Artifacts

The release includes the CLI, SDK interfaces, versioned schemas, question sets, policy files, fixtures, corpus-building documentation, evaluation report template, example repository, and a security model. It does not include a hosted service.

PRODUCT SHAPE AFTER VALIDATION

Sell trusted team workflow, not cheap tokens

Initial packaging hypothesis

Open local core

CLI, schemas, primitive interfaces, local graph store, blast-radius recipe, and evaluation harness. This drives adoption and makes the evidence chain inspectable.

Paid team control plane

Shared review queue, policy distribution, outcome tracking, repository fleet visibility, audit retention, and managed judgment budgets. Pricing is tested per active repository or usage pool, not assumed per seat.

Roadmap only after the blast-radius gate

CRAWLER	SEED	PRIMARY CUE	PROOF NEEDED
Incident	Stack trace or error	Runtime frame to source	Known incident replay
Dependency	Version bump or advisory	Imported API usage	Breaking-change corpus
Contract drift	Schema or API change	Consumers and generated clients	Cross-repo fixtures
Migration	Vocabulary or data change	Old representation references	Known incomplete migrations

Open owner decisions

1. **Repository:** Standalone public repository, or incubate privately until the precision gate?
2. **Identity:** Keep *Crawlers* as the product name, or use it as the function type inside a larger brand?
3. **Distribution:** npm package first, standalone binary first, or both from the same release?
4. **Commercial home:** Independent team tool first, or later offered as a ThatMgmt agent function?

Decisions this specification makes

Repo-local configuration, local execution, TypeScript/JavaScript first, review-only output, deterministic expansion, typed Jev judgment, a separate verifier, and a 97% precision graduation bar are defaults. Changing one requires a specification revision because it alters the experiment.

SOURCES

External evidence and market context

External claims in this specification use the sources below. Product thresholds, architecture, internal measurements, and owner decisions are stated as proposed or internal evidence, not as facts supplied by these sources.

1. Sonar. "Sonar Data Reveals Critical Verification Gap in AI Coding." Released 8 January 2026. <https://www.sonarsource.com/company/press-releases/sonar-data-reveals-critical-verification-gap-in-ai-coding/>
2. OpsLevel. "How Generative AI Is Changing Software Development: Key Insights from the DORA Report." Accessed 19 September 2026. <https://www.opslevel.com/resources/how-generative-ai-is-changing-software-development-key-insights-from-the-dora-report>
3. METR. "Early 2025 AI-Experienced Open-Source Developer Study." Published 10 July 2025. <http://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>
4. SiliconANGLE. "Greptile Bags \$25M in Funding to Take on CodeRabbit and Graphite in AI Code Validation." Published 23 September 2025. <https://siliconangle.com/2025/09/23/greptile-bags-25m-funding-take-coderabbit-graphite-ai-code-validation/>
5. Semgrep. "The Birth of Semgrep Pro Engine." 2023. <https://semgrep.dev/blog/2023/the-birth-of-semgrep-pro-engine>
6. Critique. "CodeRabbit vs Cursor Bugbot." Accessed 19 September 2026. <https://www.critique.sh/comparisons/coderabbit-vs-cursor-bugbot>
7. Vercel. "AI Gateway Zero Data Retention." Accessed 19 September 2026. <https://vercel.com/docs/ai-gateway/capabilities/zdr>
8. Vercel. "Jev Decision Models." Research note in the eve repository. Accessed 19 September 2026. <https://github.com/vercel/eve/blob/HEAD/research/jev-decision-models.md>

Evidence boundaries

The \$0.042 per million input-token price is a published vendor claim summarized in source [8]. The two measured call costs and the Jev calibration observations are internal results recorded on 17 and 19 September 2026. They are not independent benchmarks. No code-bug evaluation exists at the time of this specification.

Interpretation: The cited sources support the verification-gap, competitor-capability, pricing, and retention claims. They do not prove that Crawlers works or that teams will pay for it. The held-out experiment and design-partner study must establish those points.